Smart Garden Simulation

CSEN 275 — Computerized Garden Simulation System

Group 5 Member (Alphabetically): Jiajun Lu, Tianyu Wu, Yinmingren Fu

Agenda

01

Requirements

Scope, functional, and non-functional requirements

02

System Design

Layered architecture, control logic, and subsystems

03

GUI Design

Dashboard layout, interaction logic, and panels

04

Demo

Suggested flow and key concepts to explain

05

Future Plan

Extensions and improvements

Requirements

Grid Garden

8×14 GUI grid with configurable rows and columns. Nine plant types across fruit, vegetable, and flower categories.

Automation

Automatic watering, heating, cooling, pest control, and weather updates managed by the simulation engine.

Lifecycle Control

Start, pause, resume, stop, and speed adjustment from **1x to 10x** with real-time GUI status panels and event logging.

Key Functional Requirements

Garden & Plant Management

Place/remove plants, track growth stage, health, water level, days alive, and death conditions.

Watering System

Moisture sensors and sprinklers, skip watering during rain, manual override supported.

Temperature Control

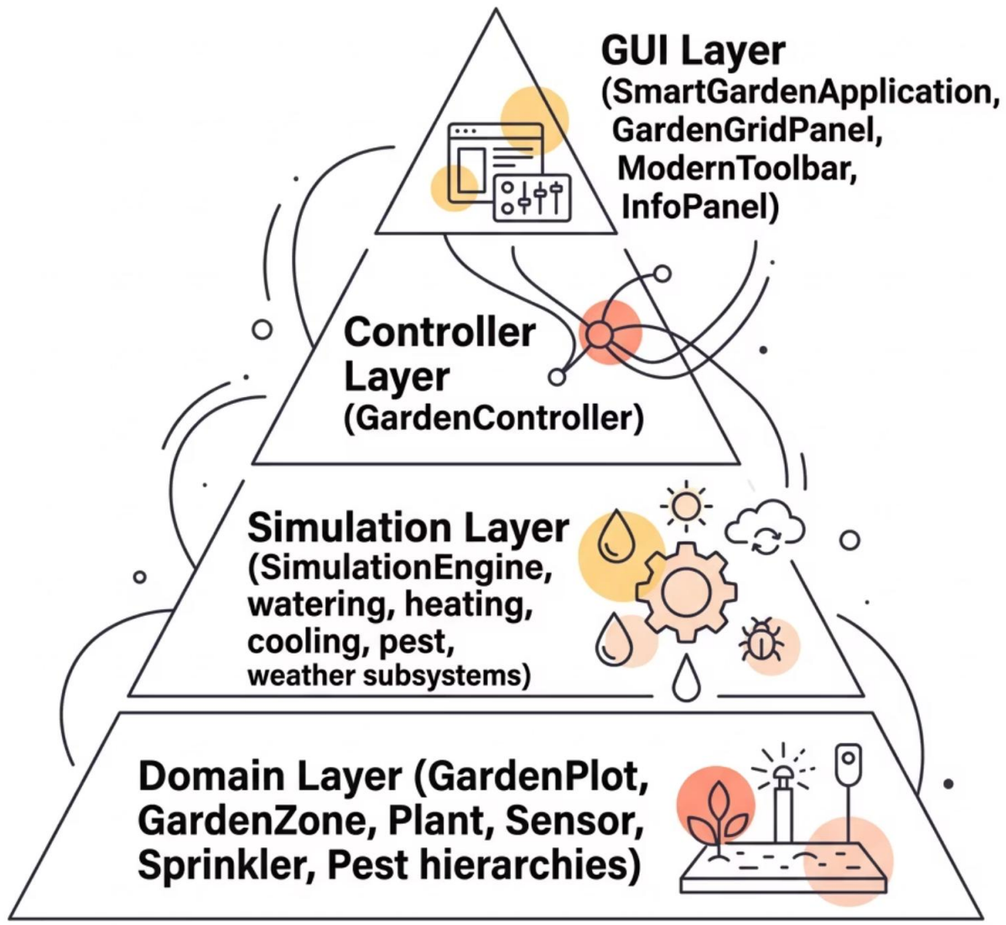
Activate heating or cooling based on zone temperature and plant-safe thresholds.

Pest Control

Spawn pests, assess zone threat levels, apply pesticide when risk is high.

Non-Functional: Smooth behavior up to 10x speed · Responsive UI · Reliable long sessions · Modular classes · Cross-platform (Windows, macOS, Linux) with Java 21 + JavaFX

System Design: Layered Architecture



Why Layered?

Interaction logic, simulation control, and domain state remain decoupled — making the project easier to explain, maintain, test, and extend.

Control Path

User → GUI → GardenController → GardenPlot / SimulationEngine → Subsystems → GUI refresh

The controller receives actions such as planting or starting the simulation, then delegates to the correct backend objects. No business logic lives in the GUI.

Simulation Engine & Subsystems

SimulationEngine is the central scheduler. On each tick it advances time, updates plants, calls every subsystem, refills low supplies, and updates counters.

💧 Watering

The watering system monitors zone moisture and plant water demand, then automatically activates sprinklers when watering is needed and rain is not occurring.

🌡️ Temperature

The temperature system continuously evaluates garden temperature and automatically applies heating or cooling to keep conditions within a plant-safe range.

🐛 Pest Control

The pest control system detects harmful pest activity, evaluates zone threat levels, and automatically applies pesticide treatment to limit plant damage.

🌦️ Weather

The weather system updates environmental conditions over time and propagates weather effects to moisture, temperature, and plant health throughout the garden.

Together, these subsystems form a **closed-loop automated garden** where environmental changes, plant health, and system actions continuously affect one another.

GUI Design

Top Toolbar

Start, Pause/Resume, Stop, speed selector, and status label

Center Grid

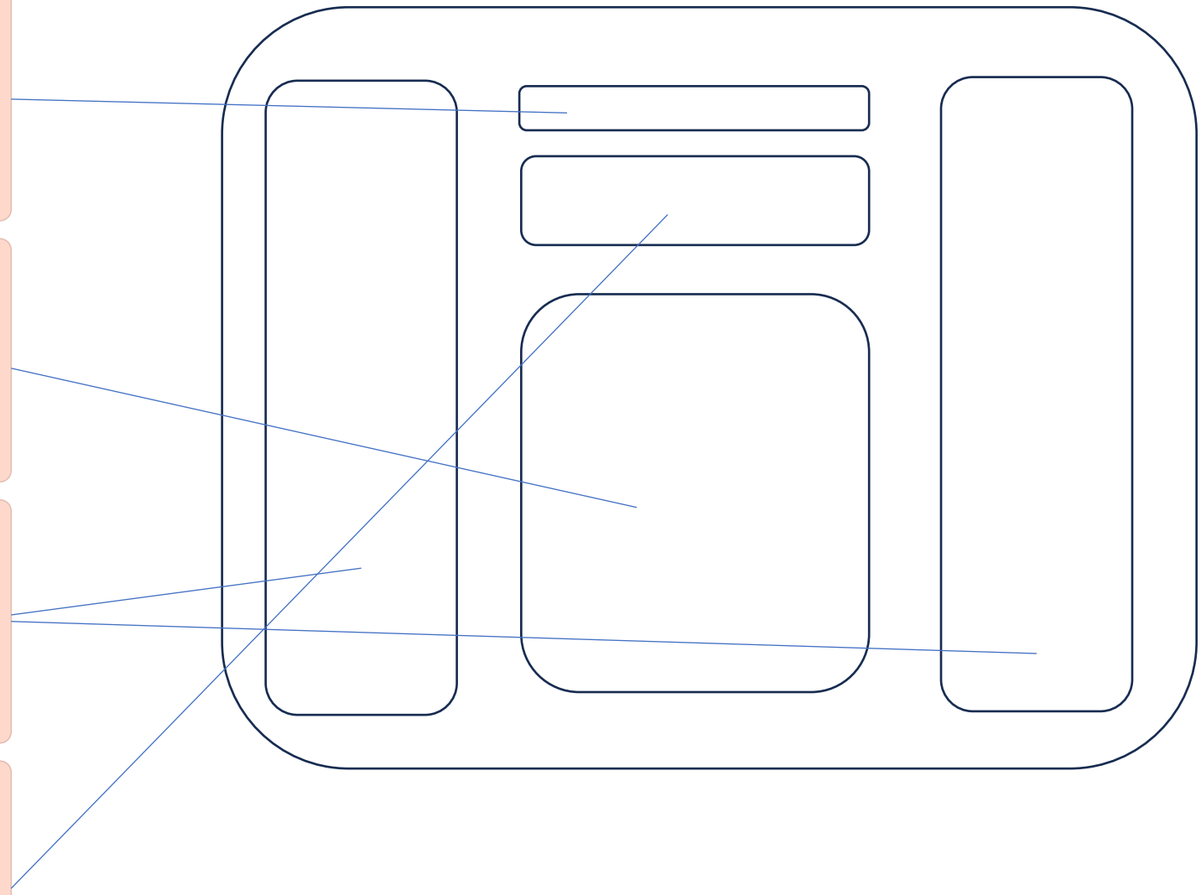
8×14 garden grid for planting and observing simulation changes

Side Panels

Left: weather, watering, temperature, pest overview. Right: day, time, inventory, plant cards

Log Panel

Recent runtime events with clear and auto-scroll controls



Future Plan

→ Save / Load Support

Preserve garden state across sessions

→ Richer Analytics

Historical charts for temperature, water usage, pest frequency, and plant survival

→ Configurable Layouts

Editable garden sizes and zone layouts beyond the default presentation grid

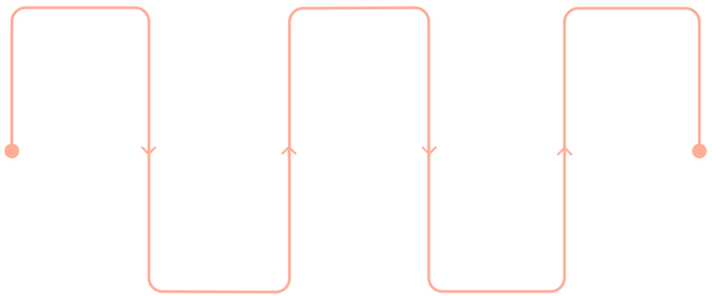
→ More Species & Behaviors

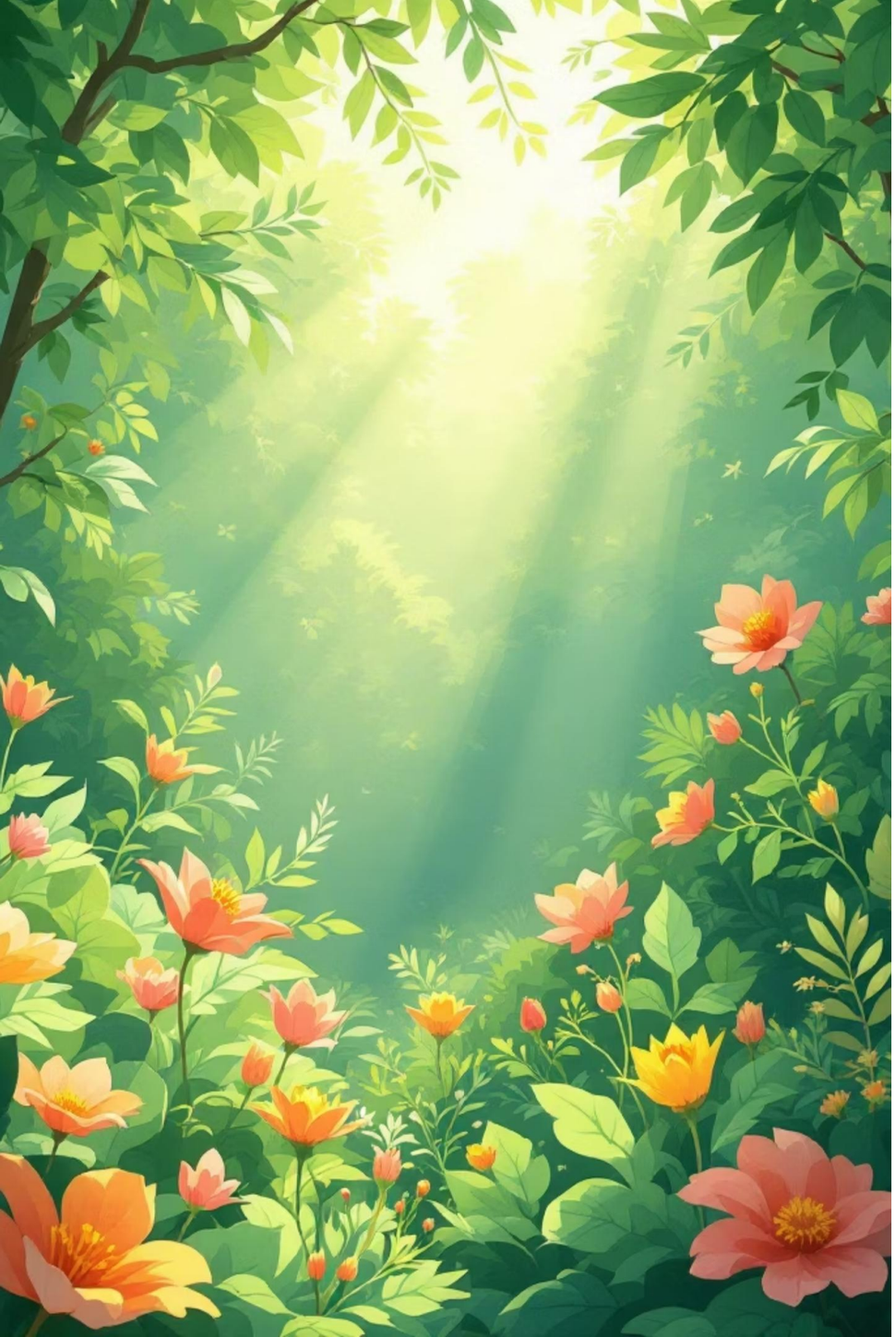
Additional plant types, realistic growth parameters, detailed pest behaviors, and stronger subsystem testing

→ Demo Mode

Dedicated presentation mode with prepared scenarios and a built-in walkthrough

System Demo





Thank You